

★ PLAYWRIGHT — INTERVIEW CHEAT SHEET

28 Questions • Answers • Practical Programs | LinkedIn-ready reference

01 PLAYWRIGHT FUNDAMENTALS

1. What is Playwright?

Answer: Playwright is an open-source browser automation and end-to-end testing framework from Microsoft. It supports Chromium, Firefox and WebKit.

Example:

```
test('example', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

2. What are the key features of Playwright?

Answer: Auto-waiting, powerful locators, cross-browser testing, parallel execution, screenshots/videos, tracing, API testing, network mocking and authentication-state reuse.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
await expect(page.getByText('Dashboard')).toBeVisible();
```

3. Which programming languages does Playwright support?

Answer: JavaScript, TypeScript, Python, Java and .NET/C#.

Example:

```
# Python
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto('https://example.com')
    browser.close()
```

4. How does Playwright differ from Selenium?

Answer: Playwright has built-in auto-waiting, BrowserContexts, tracing, network interception and a first-party test runner. Selenium is based on the WebDriver ecosystem and has a mature, broad ecosystem.

Example:

```
await page.getByRole('button', { name: 'Submit' }).click();
await expect(page.getByText('Success')).toBeVisible();
```

5. What are the main components of Playwright?

Answer: Browser, BrowserContext, Page, Locator, Playwright Test, expect assertions, APIRequestContext, fixtures and network routing.

Example:

```
Browser → BrowserContext → Page → Locator → Action → Expect
```

6. What is a BrowserContext?

Answer: An isolated browser session with separate cookies, local storage, permissions and authentication state.

Example:

```
const context = await browser.newContext();
const page = await context.newPage();
```

7. What is a Locator?

Answer: A Locator identifies an element and provides reliable actions and assertions.

Example:

```
page.getByRole('button', { name: 'Login' })
page.getByLabel('Username')
page.getByTestId('login-button')
```

8. How does auto-waiting work?

Answer: Playwright automatically waits for elements to become ready before actions. Web-first assertions also retry until the expected condition is met or the timeout expires.

Example:

```
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.getByText('Saved')).toBeVisible();
```

9. What is the purpose of expect?

Answer: expect is used for assertions such as visibility, text, URL and title validation.

Example:

```
await expect(page).toHaveURL(/dashboard/);
await expect(page.getByRole('heading')).toBeVisible();
```

10. Page vs BrowserContext?

Answer: A Page is a browser tab/document. A BrowserContext is an isolated browser session that can contain one or more Pages.

Example:

```
const context = await browser.newContext();
const page1 = await context.newPage();
const page2 = await context.newPage();
```

02 BROWSER & PAGE HANDLING

11. How do you launch and manage browsers?

Answer: Use Chromium, Firefox or WebKit. Create a context, create a page, perform actions, and close resources.

Example:

```
const browser = await chromium.launch({ headless: true });
const context = await browser.newContext();
const page = await context.newPage();
await page.goto('https://example.com');
await browser.close();
```

12. What is headless mode?

Answer: The browser runs without a visible UI. It is commonly used in CI/CD.

Example:

```
const browser = await chromium.launch({ headless: true });
```

13. How do you capture screenshots?

Answer: Use page.screenshot for a page or locator.screenshot for an element.

Example:

```
await page.screenshot({ path: 'home.png', fullPage: true });
await page.getByRole('button').screenshot({ path: 'button.png' });
```

14. How do you record test execution videos?

Answer: Configure video recording in Playwright Test, often retaining videos on failure.

Example:

```
use: { video: 'retain-on-failure' }
```

15. How do you handle multiple tabs/pages?

Answer: Wait for the context's page event while performing the action that opens the new page.

Example:

```
const newPagePromise = context.waitForEvent('page');
await page.getByText('Open Report').click();
const newPage = await newPagePromise;
```

16. How do you handle browser popups?

Answer: Use the dialog event for JavaScript alerts, confirms and prompts.

Example:

```
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept();
});
```

17. How do you work with frames/iframes?

Answer: Use frameLocator for reliable interaction inside an iframe.

Example:

```
const frame = page.frameLocator('#payment-frame');
await frame.getByLabel('Card Number').fill('4111111111111111');
```

18. How do you execute tests in parallel?

Answer: Playwright Test uses workers. Configure the worker count and keep tests independent.

Example:

```
export default defineConfig({
  workers: 4
});
```

19. How do you configure test retries?

Answer: Set retries in the configuration. Retries are useful for diagnosis but should not hide flaky tests.

Example:

```
export default defineConfig({
  retries: process.env.CI ? 2 : 0
});
```

20. How do you generate test reports?

Answer: Use built-in reporters such as HTML, list, JSON or JUnit.

Example:

```
reporter: [['html', { open: 'never' }]]
# npx playwright test
# npx playwright show-report
```

03 API • NETWORK • AUTHENTICATION

21. How do you capture API responses?

Answer: Use page.waitForResponse for a specific response or listen to response events.

Example:

```
const responsePromise = page.waitForResponse(
  r => r.url().includes('/api/users') && r.status() === 200
);
await page.getByRole('button', { name: 'Load Users' }).click();
const response = await responsePromise;
console.log(await response.json());
```

22. How do you mock API requests/responses?

Answer: Intercept the request with route and fulfill it with controlled test data.

Example:

```
await page.route('**/api/users', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([{ id: 1, name: 'Test User' }])
  });
});
```

23. How do you handle authentication?

Answer: Log in once, save storage state, and reuse it in later tests.

Example:

```
await page.getByLabel('Username').fill('testuser');
await page.getByLabel('Password').fill('password');
await page.getByRole('button', { name: 'Login' }).click();
await page.context().storageState({ path: 'auth.json' });
```

24. How do you manage cookies?

Answer: Use BrowserContext cookie APIs to add, read and clear cookies.

Example:

```
await context.addCookies([
  {
    name: 'role', value: 'admin',
    domain: 'example.com', path: '/'
  }
]);
console.log(await context.cookies());
await context.clearCookies();
```

25. How do you handle sessions and browser storage?

Answer: Use storageState to persist and reuse authentication-related cookies and local storage.

Example:

```
await context.storageState({ path: 'state.json' });
const context2 = await browser.newContext({
  storageState: 'state.json'
});
```

26. How do you test WebSockets?

Answer: Listen for WebSocket connections and inspect sent/received frames.

Example:

```
page.on('websocket', ws => {
  console.log(ws.url());
  ws.on('framereceived', data => console.log(data));
});
```

27. How do you simulate network failures?

Answer: Intercept a request and abort it or return a controlled error.

Example:

```
await page.route('**/api/payment', route =>
  route.abort('failed')
);
```

28. How do you test slow network conditions?

Answer: Delay intercepted responses and verify loading, timeout, retry and error behavior.

Example:

```
await page.route('**/api/**', async route => {
  await new Promise(r => setTimeout(r, 2000));
  await route.continue();
});
```